# Project 3
# Implementation of a Syntax Tree Generator
# **Due on Monday, December 16, 2024**

## 1. Problem

In this assignment you are requested to use the tool **Bison** to implement a syntax tree generator for the simple language **Cactus**. A library including `tree.h` and `tree.c` for syntax tree construction and printing is attached with this project.

The syntax tree generator should print the syntax tree by using the option "-t". The syntax tree generator reads input from stdin, writes output to stdout, and writes errors to stderr.

Consider the following **Cactus** program:

```
// A program to sum 1 to n
    Program
    Begin
        Var  n
        Var  s

        Read n
        If n < 0 Then
            Write -1
            Exit
        Else
            Set s = 0
        EndIf
        While n > 0 Do
            Set s = s + n
            Set n = n – 1
        EndWhile
        Write s
    End
```

The output for using the option -t is as follows:

```
(program
    ((() (var n)) (var s))
```

```
    ((((() (read n))
        (ifthenelse (< n 0)
            ((() (write (- 1))) exit)
            (() (= s 0)))
        )
        (while (> n 0)
            ((() (= s (+ s n))) (= n (- n 1))))
        )
        (write s)
    )
)
```

You don't need to handle indentation format. One long line is fine.

The executable file should be named "tree," namely,

```
flex cactus.l
bison -d cactus.y
gcc -o tree.exe lex.yy.c cactus.tab.h cactus.tab.c tree.h tree.c
type input.txt | .\tree.exe
```

To turn in this project, upload a **flex** file cactus.l and a **bison** file cactus.y to eCourse2 site.

## 3. Grading

The grading is based on the correctness of your program. The correctness will be tested by a number of test cases.

It is best to incrementally build your program so that you always have a partially-correct working program. It is also best to construct a shell script to systematically test your program.